EXP:9 – CPM ANALYSIS

AIM:

To perform Critical Path Method (CPM) Analysis for project scheduling and management using Python.

PROCEDURE:

1. Install Required Libraries:

- Make sure you have pandas, networkx, and matplotlib installed. You can install them using pip if not already installed.

  bash

  Copy code

  pip install pandas networkx matplotlib

2. Define the Problem:

- Identify the tasks, their durations, and dependencies.

3. Set Up the Problem in Python:

- Use NetworkX to represent the project tasks and their dependencies.
- Use the topological sort to identify the critical path.

4. Run the Code:

- Execute the Python code to get the critical path and project duration.

5. Analyze the Output:

- Interpret the results and validate the solution against the project schedule.

CODE:

```
import networkx as nx

import matplotlib.pyplot as plt


# Experiment: Find the Critical Path of a project using CPM (Critical Path Method)

# Tasks with durations and dependencies


tasks = {

    'A': {'duration': 3, 'dependencies': []},

    'B': {'duration': 2, 'dependencies': ['A']},
```

```
    'C': {'duration': 4, 'dependencies': ['A']},
    'D': {'duration': 2, 'dependencies': ['B']},
    'E': {'duration': 3, 'dependencies': ['C']},
    'F': {'duration': 1, 'dependencies': ['D', 'E']}
}

# Create graph
G = nx.DiGraph()

for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for dep in data['dependencies']:
        G.add_edge(dep, task)

# Topological order
topological_order = list(nx.topological_sort(G))

# Forward pass
earliest_start = {task: 0 for task in topological_order}
earliest_finish = {}

for task in topological_order:
    earliest_finish[task] = earliest_start[task] + G.nodes[task]['duration']
    for succ in G.successors(task):
        earliest_start[succ] = max(earliest_start[succ], earliest_finish[task])

# Backward pass
latest_finish = {task: earliest_finish[task] for task in topological_order}
latest_start = {}

for task in reversed(topological_order):
```

```
    latest_start[task] = latest_finish[task] - G.nodes[task]['duration']
    for pred in G.predecessors(task):
        latest_finish[pred] = min(latest_finish[pred], latest_start[task])

# Critical path
critical_path = [t for t in topological_order if earliest_start[t] == latest_start[t]]

# Short results
print("Critical Path:", " -> ".join(critical_path))
print("Project Duration:", max(earliest_finish.values()))
```

Output:

Critical Path: A -> B -> C -> D -> E -> F

Project Duration: 11

Result:

Thus, the given project scheduling problem has been solved successfully using Python and the Critical Path Method (CPM).